

Interaction System - Refactoring

1. 리팩토링 목표

기존 CInteraction_Manager::Update()는 현재 Scene에 등록된 전체 CIInteractable Registry를 매 Frame 두 차례까지 순회했다. 첫 번째 순회에서는 모든 Object의 IsInteractable()과 Priority를 검사했고, 두 번째 순회에서는 선택되지 않은 interacting Object를 정리했다.

이번 변경의 목표는 Registry를 제거하는 것이 아니라 역할을 분리하는 것이다.

- Registry: Scene, Lifetime, Script, Debug를 위한 전체 Object 목록
- Candidate: Player의 Interaction Trigger와 실제로 overlap 중인 소수 Object 목록
- Pending Request: Collision callback에서 발생한 Candidate 최종 상태
- CurrentTarget: Candidate 중 선택된 Target이며, Interaction 중에는 lifecycle 종료까지 Lock

Before 기준은 01_Baseline.md의 N=25, K=1, IsInteractable()=25.000 calls/frame, Manager Update 평균 10.233 μs, 최대 388.600 μs이다. 이 문서는 Before 문서를 수정하지 않는다.

2. 확정 설계 Decision

Decision 1. Candidate Container = std::vector

- AI가 제안한 선택지: 순회 중심 vector, 등록/제거 중심 unordered_set 또는 보조 index 구조.
- AI 추천안: 작은 K와 높은 순회 빈도를 고려한 vector.
- 개발자 검토: Candidate 등록/제거보다 매 Frame 순회가 많고, 기존 strict > Priority에서 동일 우선순위의 순회 순서를 보존해야 한다.
- 실제 코드 근거: m_Candidates는 std::vector<CIInteractable*>이고, std::find로 중복 Add를 방지하며 erase로 순서를 유지한다. 초기 reserve는 8개다.
- 최종 결정: [IMPLEMENTED] std::vector와 stable order 사용.
- 판단 결과: AI 추천안을 채택했지만, 순서 보존을 명시적인 Gameplay 조건으로 확정했다.

Decision 2. 별도 ActiveInteractionTarget 없이 CurrentTarget Lock

- AI가 제안한 선택지: 선택 Target과 실행 중 Target을 별도 Pointer로 분리하거나, 기존 CurrentTarget을 상태에 따라 Lock.
- AI 추천안: 역할이 명확한 별도 Active Pointer도 검토 가능.
- 개발자 검토: Pointer 두 개의 동기화·정리 책임이 추가되며 기존 AnimNotify가 CurrentTarget 동일성에 의존한다.
- 실제 코드 근거: Update() 시작부에서 Pending을 반영한 뒤 m_pCurrentTarget->m_bPrevInteracting이면 Candidate 선택을 수행하지 않고 Stay_Interaction()만 호출한다. Object가 실제 파괴될 때는 PurgeInteractable()이 CurrentTarget을 제거한다.
- 최종 결정: [IMPLEMENTED] 별도 Pointer를 추가하지 않고 Interaction 중 CurrentTarget Lock.
- 판단 결과: AI 선택지를 개발자가 수정하여 기존 Pointer 하나로 lifecycle을 명시했다.

Decision 3. Candidate 책임 = TriggerBox → CIInteractable → Manager

- AI가 제안한 선택지: TriggerBox가 Manager에 직접 요청하거나 Owner를 경유.
- AI 추천안: 구현 단순성을 위해 직접 요청도 가능.
- 개발자 검토: TriggerBox는 collision 사실만 전달하고, Interaction 정책과 Manager 의존은 CIInteractable에 두는 편이 책임 분리에 적합하다.
- 실제 코드 근거: CInteraction_TriggerBox는 Manager를 include하지 않는다. Player overlap 전환 시 CIInteractable::OnInteractionRangeEnter/Exit()를 호출하고, 공통 구현이 Manager의 Request API를 호출한다.
- 최종 결정: [IMPLEMENTED] Owner 경유 구조.
- 판단 결과: AI의 직접 의존 가능성을 기각하고 개발자 설계로 수정했다.

Decision 4. Pending Add / Remove를 Update 시작 시 반영

- AI가 제안한 선택지: callback 즉시 수정, double buffer, Pending Request 병합.
- AI 추천안: iterator/lifetime 안전성을 위한 Pending 방식.

- 개발자 검토: 현재 Manager Update → Collision Update_Late → 다음 Manager Update 순서에서는 다음 Update 시작 시 반영해도 기존 수준의 Frame timing을 유지할 수 있다.
- 실제 코드 근거: Request는 { pointer, final bAdd }로 동일 Object 요청을 덮어쓴다. ApplyPendingCandidates()는 Update()의 Player/Registry early-out보다 먼저 실행된다. callback이 Apply 중 새 요청을 넣어도 현재 순회를 무효화하지 않도록 Pending vector를 local vector와 swap한다.
- 최종 결정: [IMPLEMENTED] Pending 최종 상태 병합.
- 판단 결과: AI 추천안을 timing 검토 후 채택했다.

Decision 5. 전체 Registry 유지, 매 Frame Target 탐색에서는 제외

- AI가 제안한 선택지: Candidate-only 구조 또는 Registry/Candidate 분리.
- AI 추천안: Lifetime과 Scene 관리를 위해 Registry 유지.
- 개발자 검토: 최적화 대상은 Registry 자체가 아니라 매 Frame Target 탐색 범위다.
- 실제 코드 근거: m_InteractiveObjects는 등록, Purge, Clear(), Change_Scene()에 남아 있다. Update()의 Target 탐색 loop는 m_Candidates만 순회하며 기존 두 번째 Registry loop도 이전 Target 하나의 방어적 정리로 축소했다.
- 최종 결정: [IMPLEMENTED] Registry/Candidate 역할 분리.
- 판단 결과: AI 추천안을 채택했다.

3. 실제 변경 내용

CIIteractable

- 기존 구조: 순수 Interaction callback과 상태만 제공했다.
- 변경 구조: public OnInteractionRangeEnter/Exit(), protected Add/Remove helper, 공통 lifetime 정리 함수 ReleaseInteractionRegistration()을 추가했다.
- 변경 이유: 모든 파생 클래스가 Manager 요청 코드를 반복하지 않고 TriggerBox가 Manager를 직접 알지 않게 한다.
- 기존 동작 보존: 실제 Enter/Stay/Exit_InteractionRange() virtual callback은 Manager가 기존처럼 호출한다.
- 구현 조정: 요구안의 개념적 CIIteractable::Free() 이름은 다중 상속의 __super::Free()와 모호성(C2668)을 만들었다. 동일 책임을 ReleaseInteractionRegistration()으로 명명하고 공통 기반인 CInteractionObject::Free()와 CNPC::Free()에서 호출했다.

CInteraction_TriggerBox

- 기존 구조: PLAYER와 MONSTER가 m_bCollision/m_pOther 하나를 공유해 서로 상태를 덮어썼다.
- 변경 구조: m_PlayerOverlaps, m_MonsterOverlaps를 별도로 유지한다. Player의 empty/non-empty 전환만 Owner의 Candidate 진입점에 알린다.
- 변경 이유: JackyBall 하나가 Player Candidate와 Monster Grab 대상 역할을 동시에 유지해야 한다.
- 기존 동작 보존: 기존 일반 Interaction 코드의 Is_Collision()/Get_Other()는 Player accessor의 호환 alias로 유지했다. 같은 그룹 안에서는 최근 진입 대상을 반환하도록 vector의 뒤쪽 Object를 사용한다.
- Disable: Set_Active(false)는 Player Remove를 요청한 뒤 양쪽 overlap을 비운다.
- Free: Owner가 먼저 Purge한 뒤 Part가 해제되는 순서에서 dangling Pending Remove가 재생성되지 않도록, TriggerBox 파괴 경로는 Owner callback 없이 내부 overlap만 비운다.

CInteraction_Manager

- 기존 구조: 전체 Registry를 매 Frame 순회하고 Carry early return으로 Target 동일성을 우연히 유지했다.
- 변경 구조: stable Candidate vector, 병합되는 Pending Request vector, 명시적인 CurrentTarget Lock을 추가했다.
- 변경 이유: Target 탐색 비용을 전체 N이 아니라 주변 K에 비례시키고 lifecycle 의도를 코드로 표현한다.
- 주요 코드: RequestCandidateState(), ApplyPendingCandidates(), PurgeInteractable(), Candidate 기반 Update().
- 기존 동작 보존: strict > Priority, Range Enter/Stay/Exit, Interaction Enter/Stay/Exit, Scene Clear의 Registry callback 정리를 유지한다.
- Interaction 종료: non-NPC의 두 번째 A에서 Exit_Interaction() 후 m_bPrevInteracting=false로 Lock을 명시적으로 해제한다. NPC는 기존 Pressed_InteractionKey() lifecycle을 유지한다.

CInteraction_JackyBall

- 기존 구조: IsInteractable()이 Player Query와 CM_Jacky::Grab(this) Side Effect를 함께 수행했다.
- 변경 구조: IsInteractable()은 Player overlap과 기존 active/cooldown/interaction/owner 조건만 판단한다. Monster Grab은 Update_MonsterGrab()에서 별도 Monster overlap을 보고 실행한다.
- 변경 이유: 전체 Registry 순회를 제거해도 Monster Gameplay 실행 경로가 Player Candidate 여부에 종속되지 않게 하면서, JackyBall의 Player Candidate 역할은 그대로 유지한다.
- 보존한 조건: m_bActive, CanInteractive, !m_bInteraction, TriggerBox 존재, m_pOwner==nullptr, Monster overlap과 CM_Jacky type 확인. CM_Jacky::Grab() 내부의 damage/die/lift/damage-action/owner 검사는 변경하지 않았다.

CInteractionObject, CNPC

- 두 공통 Owner 계층의 Free()에서 Manager Purge를 호출한다.
- NPC Disable에서 Registry unregister를 제거했다. Disable은 Candidate만 제거하며, Registry는 Object lifetime과 재활성화를 위해 유지한다.
- Mushroom과 JackyBall에 남아 있는 기존 명시적 unregister가 호출돼도 PurgeInteractable()은 중복 호출에 안전하다.

4. Candidate 관리 구조

PLAYER Trigger Enter / Stay / Exit

- CInteraction_TriggerBox의 Player overlap 상태
- CInteractable::OnInteractionRangeEnter/Exit
- RequestAddCandidate/RequestRemoveCandidate
- CInteraction_Manager Pending 최종 상태
- 다음 CInteraction_Manager::Update() 시작 시 Apply
- std::vector Candidate K
- IsInteractable / Range lifecycle / strict '>' Priority
- CurrentTarget

Collision callback은 Candidate vector를 직접 수정하지 않는다. 동일 Frame의 Add/Remove는 마지막 요청 상태만 반영되고, Add는 Registry 등록 여부와 중복 여부를 확인한다.

5. CurrentTarget Lock

Carry를 예로 들면 다음 정책이 적용된다.

Rock Player overlap → Pending Add → Candidate → CurrentTarget

A → Enter_Interaction → m_bPrevInteracting=true → Lock

Trigger Disable → Pending Remove → Candidate에서 Rock 제거 + Range Exit

CurrentTarget은 유효한 Rock을 계속 유지 → Stay_Interaction

Interaction 종료 → m_bPrevInteracting=false → 다음 Update부터 Candidate 선택 재개

실제 Destroy → PurgeInteractable → CurrentTarget 포함 모든 raw pointer 제거

Candidate 제거는 “현재 범위 밖”을 뜻하며, 유효한 실행 중 Target의 소유권을 취소하지 않는다. 반면 실제 파괴는 lifetime 안전성이 우선이므로 Lock 여부와 무관하게 Target pointer를 제거한다.

개발자가 직접 수행한 검증에서 Carry Range, UI, Carry State, PlayerOnCarry AnimNotify, Rock Attach, Throw는 [PASS]였다. CurrentTarget identity와 나머지 세부 lifecycle 항목은 별도 결과가 없으므로 [NOT VERIFIED]로 유지한다.

6. Lifetime 관리

상황	Registry	Candidate	Pending	CurrentTarget / Range
Range Enter	유지	다음 Update에 stable Add	Add 병합	Candidate 평가 후 Range Enter 가능

상황	Registry	Candidate	Pending	CurrentTarget / Range
Range Exit	유지	다음 Update에 stable erase	Remove 병합	Range Exit 호출, interacting Target은 Lock 유지
Disable	유지	TriggerBox가 Remove 요청	최종 상태 병합	Player overlap 해제, Monster overlap도 별도 정리
Free	Purge	즉시 제거	해당 Pointer 요청 제거	Interaction/Range 정리 후 Target Pointer 제거
Destroy	명시 unregister와 공통 Free 모두 idempotent	제거	제거	dangling CurrentTarget 방지
Scene/Room Clear	Registry callback 후 재구성	clear	clear	Current/Previous Target clear

Room cache 비활성화/재활성화의 실제 Collision 재참여는 [NOT VERIFIED]이다.

7. 구현 과정에서 발견한 문제와 해결

7.1 JackyBall Query에 숨겨진 Monster Gameplay Side Effect

문제 상황: → 전체 Registry 순회가 Player Target 탐색뿐 아니라 JackyBall Monster Grab Gameplay까지 암묵적으로 실행하고 있었다. JackyBall은 Player와 Monster 모두가 동일 Object에 상호작용한다.

원인 분석: → CInteraction_JackyBall::IsInteractable() Query 함수에 Player 가능 여부 판단과 CM_Jacky::Grab(this) Gameplay Side Effect가 결합돼 있었다.

위험: → Candidate 기반 순회로 변경하면 Player Candidate가 아닌 JackyBall의 IsInteractable() 호출이 사라져 Monster Gameplay Regression 가능성이 존재했다. 반대로 JackyBall을 Player Candidate에서 제외하면 Player Grab 경로가 손상된다.

검토한 해결 방법: → 전체 Registry 순회를 다시 유지하는 방법, JackyBall 역할 자체를 분리하는 방법, 하나의 Object에서 실행 책임만 분리하는 방법을 검토했다.

최종 해결: → JackyBall 자체의 두 역할은 유지했다. Player overlap은 Candidate 기반 IsInteractable() Query로, Monster overlap은 JackyBall 자체 Update의 Update_MonsterGrab()으로 처리한다. PLAYER/MONSTER overlap은 TriggerBox에서 서로 덮어쓰지 않게 분리했다. Manager는 Candidate 기반 Target 탐색만 수행한다.

결과: → 개발자 검증에서 Player Grab과 Monster Grab은 각각 [PASS]였다. Player와 Monster가 동시에 같은 Ball의 ownership을 경쟁하는 경우는 별도로 검증됐다고 단정하지 않고 [NOT VERIFIED]로 유지한다.

7.2 공통 Free 이름과 다중 상속 충돌

문제 상황: → CIIInteractable::Free()를 추가하자 CInteractionObject의 __super::Free()가 두 직접 기반의 동명 함수 사이에서 모호해졌다(C2668).

원인 분석: → CInteractionObject와 CNPC는 Engine GameObject 계층과 CIIInteractable을 다중 상속한다.

검토한 해결 방법: → 모든 파생 Free를 직접 한정하는 방법과 정책 함수 이름을 역할 중심으로 분리하는 방법을 검토했다.

최종 해결: → 공통 정책 함수명을 ReleaseInteractionRegistration()으로 바꾸고 두 공통 Owner 계층에서 호출했다.

결과: → 다중 상속의 기존 __super::Free() 관례를 유지하면서 Client Debug x64 링크가 성공했다.

7.3 Part Free에서 Pending Pointer 재생성 가능성

문제 상황: → Owner가 Purge된 뒤 TriggerBox Part의 Free가 Owner Exit callback을 다시 호출하면 파괴 중인 Owner Pointer가 Pending에 재등록될 수 있었다.

원인 분석: → normal Disable과 destruction이 동일 overlap-clear helper를 사용하면 destruction 순서가 Owner life-time보다 늦다.

최종 해결: → normal Set_Active(false)는 Remove를 요청하지만, TriggerBox Free()는 Owner callback 없이 내부 overlap vector만 비운다. Manager raw pointer 정리는 Owner의 공통 Purge가 담당한다.

결과: → 파괴 경로에서 Pending dangling pointer를 다시 만들지 않는 구조로 정리했다.

7.4 Build 환경/산출물 이슈

- Solution 병렬 Build 중 MapTool의 기존 CLevel_Loading.obj에 LNK1136이 발생했다.
- Client 병렬 Build는 프로젝트의 shared PDB 설정 때문에 C1041이 발생했다.
- Client를 단일 프로세스(/m:1, multi-processor compile off)로 빌드해 리팩토링 Source의 실제 오류를 분리했고, include 및 함수명 문제를 수정한 뒤 Client.exe 링크를 완료했다.

8. Regression Test 결과

Gameplay 결과는 개발자가 직접 수행한 검증 보고를 반영했다. 명시적으로 보고되지 않은 항목은 추론하지 않았다.

항목	상태
Logo → Town	[NOT VERIFIED]
Rock Interaction Range	[PASS]
Interaction UI	[PASS]
CurrentTarget = 최초 Rock	[NOT VERIFIED]
A Input	[NOT VERIFIED]
Carry State	[PASS]
Carry Animation	[NOT VERIFIED]
PlayerOnCarry AnimNotify	[PASS]
AnimNotify Target 동일성	[NOT VERIFIED]
Rock Attach	[PASS]
Holding	[NOT VERIFIED]
두 번째 A	[NOT VERIFIED]
Throw	[PASS]
정상 상태 복귀	[NOT VERIFIED]
Player가 JackyBall Grab	[PASS]
Monster가 JackyBall Grab	[PASS]
Player/Monster가 동일 Ball을 대상으로 하는 기존 규칙	[NOT VERIFIED]
Room cache 재활성화	[NOT VERIFIED]

9. Before / After

After benchmark는 기존 계측 기능만 사용해 다음 조건으로 측정했다.

- Build/실행 파일: Debug | x64, Client/Bin/Client.exe
- Scene/위치: Town, 기존 F10 Rock checkpoint (11.0471277, 12.8000002, 65.7022171)
- 구간: latestK=minK=maxK=avgK=1이 유지된 F8 Reset 이후 600 Frame
- 계측: 기존 Interaction_Baseline.log의 자동 600 Frame dump
- 환경 제어: 개발자 요청에 따라 Town의 Debug R → Boss 단축 전환을 주석 처리해 측정 중 Town을 유지했다. Interaction Manager 계측 코드와 Gameplay 코드는 추가하지 않았다.

항목	Before	After	상태/변화
Registry N	25	25	유지
Candidate K	1 (기존 InRange)	1.000	유지, 전 구간 1
IsInteratable Calls / Frame	25.000	1.000	24회/Frame 감소, -96.000%
Priority Compare / Frame	0.000	0.000	단일 Candidate이므로 동일

항목	Before	After	상태/변화
Manager Update Avg	10.233 μ s	8.945 μ s	-1.288 μ s, -12.587%
Manager Update Max	388.600 μ s	85.800 μ s	-302.800 μ s, -77.921%
Measurement Frames	600	600	동일

After 원본 dump:

[InteractionBaseline] frames=600 latestN=25 maxN=25 avgN=25.000 latestK=1 minK=1 maxK=1 avgK=1.000 isCallsPerFrame=1.000

Debug microsecond 값에는 scheduling noise가 포함될 수 있으므로 실행 시간 감소율만으로 성공을 단정하지 않는다. 직접적인 구조 성공 지표는 N=25가 유지되는 동안 K=1과 IsInteratable=1.000/frame이 일치했다는 점이다. Collision Manager 값은 비교에서 제외했다.

10. 최종 결과

- Client Debug x64 Build: [PASS]
- 전체 Registry 유지: [IMPLEMENTED]
- Registry의 매 Frame Target 탐색/두 번째 전체 순회 제거: [IMPLEMENTED]
- Candidate K 기반 선택, Pending 병합, CurrentTarget Lock: [IMPLEMENTED]
- Player/Monster overlap 및 JackyBall Query/Side Effect 책임 분리: [IMPLEMENTED]
- 개발자가 명시한 Carry 6개 항목 및 JackyBall Player/Monster Grab: [PASS]
- N→K 기반 Target 탐색: [MEASURED] (N=25, K=1, IsInteratable=1.000/frame)
- Manager Update: [MEASURED] (평균 8.945 μ s, 최대 85.800 μ s, 600 Frame)

명시적으로 검증되지 않은 CurrentTarget identity, Carry Animation/Holding/정상 복귀, 동시 Player/Monster ownership, Room cache 재활성화는 계속 [NOT VERIFIED]이다.